

Sequential Decision-making under Uncertainty

Nan Ye

School of Mathematics and Physics
University of Queensland

“Mama always said that **sequential decision making under uncertainty**
life was like a box of chocolates.
You never know what you're gonna get.”

Forrest Gump



optimal decisions are often elusive (analytically and computationally)
good decisions are often computable (via iterative methods)

Sequential Decision Making



ad placement



play Go (DeepMind)



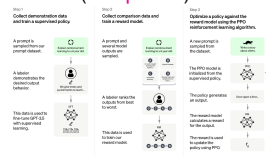
play StarCraft (DeepMind)



robot control (Boston Dynamics)



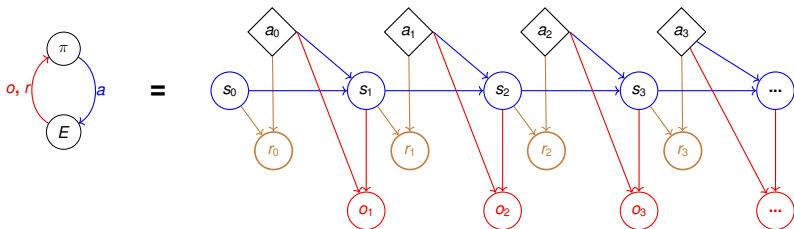
autonomous car (Waymo)



ChatGPT (OpenAI)

many others: algorithm discovery, ride-hailing order dispatching, dialogue systems, healthcare, energy, ...

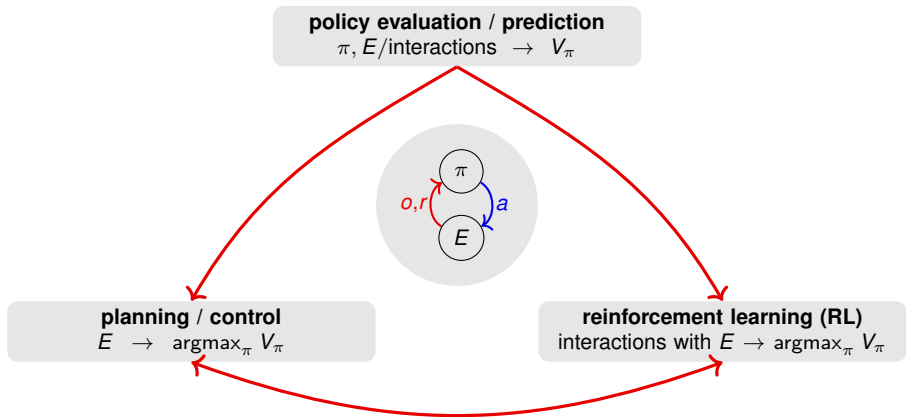
General framework



an agent π interacting with an environment E
at each time step, agent executes an action a , receives an observation o and a reward r , and
environment state evolves

Special cases

- Bandits: stateless, only receive rewards
- MDPs (Markov Decision Processes): evolving fully observed states
- Partially Observable MDPs (POMDPs): evolving partially observed states



π = policy, V_π = policy value, E = environment, o = observation, r = reward

three interconnected problems

Iterative Refinement

Exact analytical solutions are rarely available.

Many solution methods are computational and they iteratively refine a solution.

An iterative method can be written in the form $x_{t+1} = H_t(x_t)$

- x is the solution
 - this may be a policy or its value or both
 - common representations: tables, parametric functions (e.g., neural networks), trees
- refinement operator H_t uses environment/interactions
 - this may be deterministic or stochastic
 - this may be defined explicitly or implicitly

This Talk

- Sequential decision-making under uncertainty
 - examples, general framework, special cases (bandits, MDPs, POMDPs)
 - three problems (evaluation, planning, RL)
 - iterative refinement
- Theory and algorithms
 - bandits: regret minimization, UCB1, ϵ_t -greedy
 - MDPs: policy evaluation (iterative, linear method), planning (value iteration, Q-iteration), RL
 - POMDPs: belief tree search, DESPOT, ADVT, LCEOPT

Multi-armed Bandits (MABs)

What's the best sequence of pulls for K bandits (slot machines)?



A prototypical RL problem, often used in solving MDPs/POMDPs

Many other similar problems (Bouneffouf and Rish, 2019)

ad placement, clinical trials, dynamic pricing, recommender systems, algorithm selection, ...

Various formulations

stochastic bandits, adversarial bandits, Markovian bandits, contextual bandits

- We focus on stochastic bandits satisfying the following assumptions
 - fixed but unknown reward distributions with means $\mu_1, \dots, \mu_K$
 - for each pull, a reward is sampled from the pulled arm's distribution, independently from the past
 - bounded rewards in $[0, 1]$
- Ideal (non-achievable) strategy: pull the arm with the highest mean reward
- Practical strategy: need to explore (try less played arms) and exploit (play rewarding arms).
- Solution methods generally follow an iterative refinement structure
 - they repeatedly select an arm to play (hard) and update reward statistics (easy)

Regret Minimization

- We would like to play to minimize the expected regret

$$R_T = T\mu_* - \mathbb{E}\left[\sum_{t=1}^T r_t\right],$$

where T is the number of pulls, $\mu_* = \max_i \mu_i$, and r_t is the reward at time step t .

- Alternatively, the expected regret is

$$R_T = T\mu_* - \sum_{k=1}^K \mu_k \mathbb{E}[n_k(T)],$$

where $n_k(T)$ is the number of pulls for k at time step T .

- Lower bound: R_T is at least of the order $O(\ln T)$ (Lai and Robbins, 1985).

Upper Confidence Bound (UCB)

- UCB1: at each time step t , play machine j with maximum

$$\bar{x}_j + \sqrt{\frac{2 \ln t}{n_j}},$$

where $\bar{x}_j$ =average reward for machine j , and n_j = number of plays for machine j .

- an example of optimism in the face of uncertainty
 - each arm is played at least $c \ln t$ times at time t for some $c > 0$
- The expected regret of UCB1 is

$$R_T \in O(\ln T),$$

where $\Delta_k = \mu_* - \mu_k$, and μ_k is the expected reward for machine k .

ϵ_t -greedy

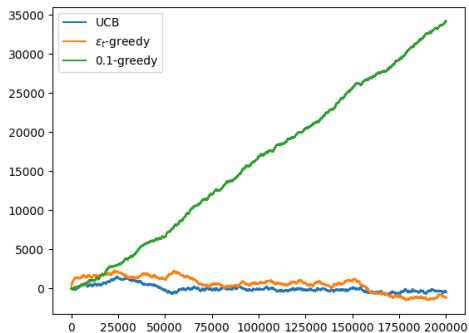
- ϵ_t -greedy: at each time step t , play $j^* = \operatorname{argmax}_j \bar{x}_j$ w.p. $1 - \epsilon_t$, and play a random arm w.p. ϵ_t , where

$$\epsilon_t = \min \left(1, \frac{cK}{d^2 t} \right),$$

with a suitable $d > 0$ and any $c > 0$.

NB Constant ϵ_t does not work.

- The probability of pulling a suboptimal arm is $O(1/t)$ at time step t
 $\Rightarrow R_T \in O(\ln T)$



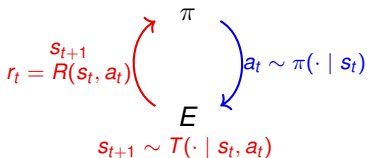
$T\mu_* - \sum_{t=1}^T r_t$ against t in a simulation

Markov Decision Processes (MDPs)

MABs are stateless, some problems have states $\Rightarrow$ MDP.
State: useful environment information for making decisions.

MDP (S, A, T, R, γ)

- state space S
- action space A
- transition model $T(s' | s, a)$
- reward function $R(s, a)$
- discount factor $\gamma \in [0, 1]$

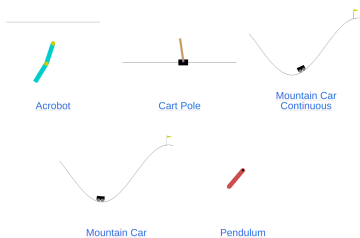


Objective: find an optimal (stochastic) policy $\pi : S \rightarrow \Delta_A$ such that

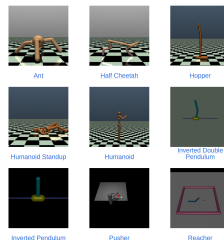
$$V_\pi(s) \geq V_{\pi'}(s) \text{ for any } s \text{ and any policy } \pi' : S \rightarrow \Delta_A$$

where $V_\pi(s) = \mathbb{E}(\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, \pi)$ is the value of π starting from s , and Δ_A denotes probability distributions on A .

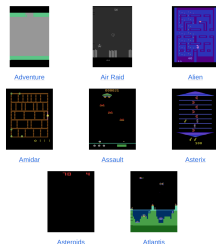
Remarks. (a) often exists an optimal deterministic policy $\pi : S \rightarrow A$. (b) often interested in finite horizon problems too. (c) lots of variants and highly technical results.



classic control



MuJoCo



Atari

DACBench: a benchmark for Dynamic Algorithm Configuration

Continued: 48.2% 0 items 47

A benchmark library for **Dynamic Algorithm Configuration**. Its focus is on reproducibility and comparability of different DAC methods as well as easy analysis of the optimization process.

flappy-bird-env

Continued: 48.2% 0 items 48

Flappy Bird as a Paramo Gymnasium environment.

flappy-bird-gymnasium: A Flappy Bird environment for Gymnasium

Continued: 48.2% 0 items 49

A simple environment for single-agent reinforcement learning algorithms on a clone of **Flappy Bird**, the hugely popular arcade-style mobile game. Both state and pixel observation environments are available.

gym-cellular-automata: Cellular Automata environments

Continued: 48.2% 0 items 50

ON THIS PAGE

DACBench: a benchmark for Dynamic Algorithm Configuration
 flappy-bird-env
 flappy-bird-gymnasium: A Flappy Bird environment for Gymnasium
 gym-cellular-automata: Cellular Automata environments
 gym-jimmy: Training Robots in Jimmy
 gym-nature: Environments used to prove theorems
 gym-robotics: Training Environment
 highway-env: Autonomous driving and tactical decision-making tasks
 mario-mip: Easily create discrete MDPs
 mobile-env: Environments for coordination of wireless mobile networks
 pendulum: Robotics environments using the PyBullet physics engine
 Safety-Gymnasium: Ensuring safety in real-world RL scenarios
 stable-baselines: Classic reinforcement learning, a maintained version of OpenAI Baselines

many 3rd party environments

many publicly available benchmark environment simulators

Policy evaluation

- The value function V_π of a policy π satisfies the Bellman equation

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_{\substack{a|s \sim \pi \\ s'|s, a \sim T}} (R(s, a) + \gamma V_\pi(s')) \\ &= \sum_a \pi(a | s) \sum_{s'} T(s' | s, a) (R(s, a) + \gamma V_\pi(s')) . \end{aligned}$$

- Equivalently, V_π is the fixed point of the Bellman operator H_π :

$$V_\pi = H_\pi(V_\pi),$$

where the Bellman operator $H : \mathbb{R}^S \rightarrow \mathbb{R}^S$ is defined by

$$H(V)(s) \stackrel{\text{def}}{=} \mathbb{E}_{\substack{a|s \sim \pi \\ s'|s, a \sim T}} (R(s, a) + \gamma V(s')) ,$$

- Iterative policy evaluation (fixed-point iteration for H_π)
 - Choose arbitrary V_0 , compute $V_{t+1} = H_\pi(V_t)$ for $t \geq 0$.
 - V_t converges to V_π (proved using Banach fixed-point theorem).
- Linear method: $V_\pi = H_\pi(V_\pi)$ is a linear system $\Rightarrow$ use linear solvers

Planning

- The optimal value function V^* satisfies the Bellman optimality equation

$$V^*(s) = \max_a \mathbb{E}_{s' | s, a \sim T} (R(s, a) + \gamma V^*(s')) .$$

- Equivalently, V^* is the fixed point of the Bellman operator H :

$$V^* = H(V^*),$$

where the Bellman operator $H : \mathbb{R}^S \rightarrow \mathbb{R}^S$ is defined by

$$H(V)(s) \stackrel{\text{def}}{=} \max_a \left(\sum_{s'} T(s' | s, a) (R(s, a) + \gamma V(s')) \right) ,$$

- Value iteration (fixed-point iteration for H)
 - Choose arbitrary V_0 , compute $V_{t+1} = H(V_t)$ for $t \geq 0$.
 - V_t converges to V^* (proved using Banach fixed-point theorem).
- Given $V \approx V^*$, we can compute a good policy

$$\pi(s) = \operatorname{argmax}_a \left(R(s, a) + \gamma \sum_{s'} T(s' | s, a) V(s') \right)$$

- Often easier to work with *action-value function* $Q_\pi(s, a)$, defined as

$$Q_\pi(s, a) = \mathbb{E}(\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a, \pi).$$

- Bellman equation for the optimal action value function Q^* :

$$Q^*(s, a) = \mathbb{E}_{s' \mid s, a \sim T} \left(R(s, a) + \gamma \max_{a'} Q^*(s', a') \right).$$

- Equivalently, Q^* is the fixed point of the Bellman operator H (*overloaded notation!*):

$$Q^* = H(Q^*),$$

where the Bellman operator $H : \mathbb{R}^{S \times A} \rightarrow \mathbb{R}^{S \times A}$ is defined by

$$H(Q)(s, a) \stackrel{\text{def}}{=} \mathbb{E}_{s' \mid s, a \sim T} \left(R(s, a) + \gamma \max_{a'} Q(s', a') \right),$$

- Q-iteration (fixed-point iteration for H)
 - choose an arbitrary Q_0 , and compute $Q_{t+1} = H(Q_t)$
 - Q_t converges to Q^*
- Given $Q \approx Q^*$, we can compute a good policy

$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$

Properties of optimal policies

- Optimal policies are monotonic for a class of deterministic MDPs motivated by fishery management (joint work with Ju, Filar & Kroese)
 - s transitions to $\zeta(s, a)$ upon executing an action a .
 - $\zeta(s, a)$ is strictly increasing in s and strictly decreasing in a .
 - $R(s, a)$ is strictly increasing in both s and a .
 - $R(s, a)$ and $\zeta(s, a)$ are both strictly concave in s
 - $\frac{\partial R}{\partial a}(s, a) + c \frac{\partial \zeta}{\partial a}(s, a) = 0$.
- Optimal harvesting strategies for non-stationary deterministic MDPs (joint work with Bohner & Filar)
 - Population grows from x_t to $f_t(x_t)$ at time t .
 - Harvesting with a survival rate γ_t : $x_{t+1} = \gamma_t f_t(x_t)$.
 - Find harvest rates maximizing the total yield
 $(1 - \gamma_1)f_1(x_1) + \dots + (1 - \gamma_n)f_n(x_n)$
 - An optimal strategy has a tendency to achieve $f'_t(x_t) = 1$.

Reinforcement Learning (RL)

- RL for MDPs often builds on ideas in planning
- Model-based approaches: learn an MDP model using the interactions, and use planning to compute an optimal for the learned model.
- Model-free approaches: often turns an exact steps in planning to stochastic steps based on interactions
 - Q-iteration: $Q \leftarrow H(Q_t)$
 - Q-learning: $Q \leftarrow \tilde{H}(Q_t)$, where $\tilde{H}$ is a stochastic approximation of H that performs the following update if we experience a transition (s, a, s', r) :

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

where $\alpha > 0$ is a learning rate.

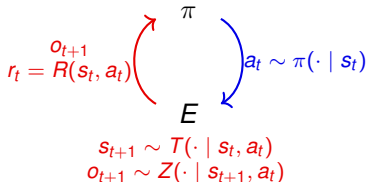
- Numerous works on RL in recent years, with applications in robot control, large language models, discovering fast matrix multiplication algorithms...

POMDPs

Partially Observable Markov Decision Processes

POMDP $(b_0, S, A, O, T, Z, R, \gamma)$

- initial state distribution b_0
- state/action/observation space $S/A/O$
- transition model $T(s' | s, a)$
- observation function $Z(o | s', a)$
- reward function $R(s, a)$
- discount factor $\gamma \in [0, 1]$



Objective: find an optimal policy $\pi : \Delta_S \rightarrow A$ such that

$$V_\pi(b) \geq V_{\pi'}(b) \text{ for any belief } b \text{ and any policy } \pi' : \Delta_S \rightarrow A$$

where $V_\pi(b) = \mathbb{E}(\sum_{t=0}^{\infty} \gamma^t r_t | s_0 \sim b_0, \pi)$ is the value of π starting from initial belief b , and Δ_S is the probability distributions on A .

A POMDP is equivalent to an MDP with beliefs as states, called the belief MDP.

Planning is hard: finite-horizon POMDP is NP-hard, infinite-horizon POMDP is undecidable.

Algorithms are scaling up

- 1990's: tens of states is challenging.
- 2000's: offline point-based methods
PBVI (thousands of states), SARSOP (hundreds of thousands of states)
- after 2010: online belief tree search
POMCP, GPS-ABT, DESPOT, POMCPOW, ADVT, LCEOPT, ...

Offline algorithms plan for **all** beliefs

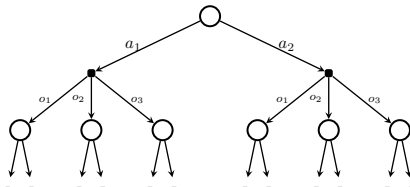
- Plan first, execute later.
- Planning is often slow, but execution is often fast.
- Generally do not exploit the local structure in the search space.

Online algorithms plan for **current** belief

- Interleave planning and execution.
- Possibility of benefiting from local structure in the search space.

Belief-tree Search

- A belief tree encapsulates all possible strategies

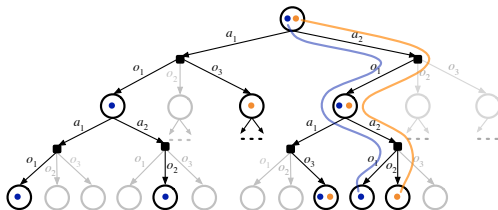


- root node is the initial belief
- following an action a from a belief node b and observing o leads to a belief node representing the updated belief
- Naive belief tree search
 - truncate the tree to a finite horizon
 - estimate values of leaf beliefs
 - compute actions values in a bottom-up manner
- Many online algorithms construct an iteratively refined approximation to the belief tree

DESPOT

Joint work with Somani, Hsu and Lee

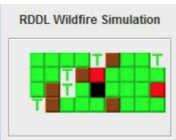
- An anytime online Monte Carlo algorithm for POMDPs with discrete actions and observations.
 - Evaluate policies on *shared scenarios*, penalizing large policies.



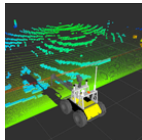
- Provably near-optimal.
- Good empirical performance



self-driving



IPPC'14 winner



HRATC'15 winner

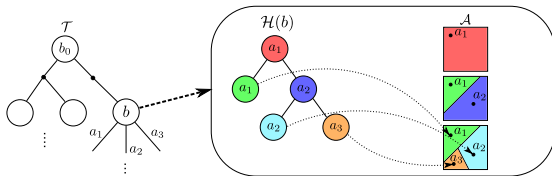
ADVT

Joint work with Hoerger, Kurniawati and Kroese

- An anytime online Monte Carlo algorithm for POMDPs with continuous actions.
- Example robotic tasks



Idea

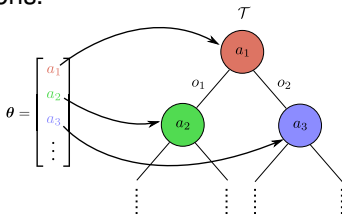


- maintains an adaptively refined hierarchical partition, called Voronoi tree, of the action space for each node
- starts with a single belief node tree, and performs simulations to grow it to better approximations for the belief tree
- for each simulation, each step chooses an action from the representative actions in the Voronoi tree using a modified UCB1 strategy
- a cell in a Voronoi tree is randomly split whenever it has been sufficiently explored
- continuous observations are handled using a progressive widening strategy that samples a new observation when current observations have been sufficiently explored.

LCEOPT

Joint work with Hoerger, Kurniawati and Kroese

- An anytime online algorithm for POMDPs with continuous actions and discrete observations.



- solve $\max_{\pi} V_{\pi}(b)$ over the space of policies represented as policy trees
- each policy tree π can be parametrized as a vector θ , thus we are solving a problem of the form $\max_{\theta} V(\theta)$
- naively, this can be solved directly using the cross-entropy (CE) method
- evaluating a policy π by simulation often does not require knowing all of π 's parameters
 $\Rightarrow$ we only keep the encountered parameters and develop a highly efficient lazy CE method

This Talk

- Sequential decision-making under uncertainty is ubiquitous
 - bandits, MDPs and POMDPs are prototypical models with many applications
 - we often need to consider three interconnected problems: policy evaluation, planning, and RL
- Exact analytical solutions are rarely available.
- Many solution methods are computational and they iteratively refine a solution.
 - Bandits: UCB1, ϵ_t -greedy
 - MDPs: iterative policy evaluation, value iteration, Q-iteration, RL
 - POMDPs: DESPOT, ADVT, LCEOPT